



RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science and Engineering

Lab Project Report - "Compiler Design"

Submitted by

Md Naim Parvez
Roll No: 2003015
Section: A
4th Year (Odd Semester)

Submitted to

Farjana Parvin
Assistant Professor
Department of CSE
Rajshahi University of Engineering &
Technology

Course Code: 4102

Course Title: Compiler Design Sessional

Contents

1	Introduction	1
2	Literature Review	1
3	Implementation	2
4	Output Analysis	2
5	Conclusion	5
A	Complete Source Code	6
A.1	Lexer.l - Lexical Analyzer	6
A.2	Parser.y - Syntax Analyzer	7

1 Introduction

Overview

Compiler design is a fundamental area of computer science that bridges high-level programming languages and machine-level execution. This project implements a complete compiler for a custom programming language with Bengali-inspired keywords, demonstrating the entire compilation pipeline from source code to assembly language.

Objectives

The primary objectives of this project are:

- Design and implement a lexical analyzer using Flex
- Develop a parser using Bison for syntax analysis
- Create a symbol table for variable management
- Generate intermediate code representation
- Produce x86 assembly code output

2 Literature Review

Compiler Structure

A compiler typically consists of several phases working in sequence. The front-end phases include lexical analysis, syntax analysis, and semantic analysis, while the back-end phases handle intermediate code generation, optimization, and target code generation. This project implements the essential phases required for a functional compiler.

Lexical Analysis

Lexical analysis is the first phase of compilation, where the source code is read as a stream of characters and grouped into meaningful sequences called tokens. Tools like Flex (Fast Lexical Analyzer) automate this process by using regular expressions to define token patterns.

Syntax Analysis

Syntax analysis, or parsing, verifies that the token sequence conforms to the language's grammar rules. Parser generators like Bison use context-free grammars to build parse trees and detect syntax errors. The parser works in conjunction with the lexer to build a hierarchical representation of the program structure.

Code Generation

Code generation translates the intermediate representation into target machine code. For this project, x86 assembly language serves as the target, requiring careful management of registers, memory, and the runtime stack.

3 Implementation

input.txt

```
1 FUNC main ()
2 start
3     mone_kori a <-- integer
4     mone_kori b <-- integer
5     a equal 12
6     b equal 13 jog a
7     print_kori b
8 end
```

Listing 1: Input Test Program (Input.txt)

4 Output Analysis

Lexical Analysis Output

The lexer successfully tokenizes the input:

```
1 Token: FUNC
2 Token: ID (main)
3 Token: (
4 Token: )
5 Token: start
6 Token: mone_kori
7 Token: ID (a)
8 Token: <--
9 Token: integer
10 Token: mone_kori
11 Token: ID (b)
12 Token: <--
13 Token: integer
14 Token: ID (a)
15 Token: equal
16 Token: ICONST (12)
17 Token: ID (b)
18 Token: equal
19 Token: ICONST (13)
20 Token: jog
21 Token: ID (a)
22 Token: print_kori
23 Token: ID (b)
24 Token: end
```

Listing 2: Lexical Analysis Output

```

output.txt
1  ===== LEXICAL ANALYSIS =====
2  Token: FUNC
3  Token: ID (main)
4  Token: (
5  Token: )
6  Token: start
7  Token: mone_kori
8  Token: ID (a)
9  Token: <--
10 Token: integer
11 In line no 3, Inserting a with type INT_TYPE in symbol table.
12 Token: mone_kori
13 Token: ID (b)
14 Token: <--
15 Token: integer
16 In line no 4, Inserting b with type INT_TYPE in symbol table.
17 Token: ID (a)
18 Token: equal
19 Token: ICONST (12)
20 Token: ID (b)
21 Token: equal
22 Token: ICONST (13)
23 Token: jog
24 Token: ID (a)
25 Token: print_kori
26 Token: ID (b)
27 Token: end
28

```

Figure 1: Lexical Analysis Output

Syntax Analysis Output

The parser generates the parse tree:

```

1  statements -> empty
2  declaration -> MONE_KORI ID TYPE_ASSIGN INTEGER
3  statement -> declaration
4  statements -> statements statement
5  declaration -> MONE_KORI ID TYPE_ASSIGN INTEGER
6  statement -> declaration
7  statements -> statements statement
8  exp -> ICONST
9
10 assignment_print_scan -> ID EQUAL exp
11 statement -> assignment_print_scan
12 statements -> statements statement
13 exp -> ICONST
14 exp -> ID
15 exp -> exp JOG exp
16
17 assignment_print_scan -> ID EQUAL exp
18 statement -> assignment_print_scan
19 statements -> statements statement
20 assignment_print_scan -> PRINT_KORI ID
21 statement -> assignment_print_scan
22 statements -> statements statement
23 func_def -> FUNC ID LPAREN RPAREN STARTT statements END
24 program -> func_def

```

Listing 3: Parse Tree

```

30
31 ===== SYNTAX ANALYSIS =====
32 statements -> empty
33 declaration -> NONE_KORI ID TYPE_ASSIGN INTEGER
34 statement -> declaration
35 statements -> statements statement
36 declaration -> NONE_KORI ID TYPE_ASSIGN INTEGER
37 statement -> declaration
38 statements -> statements statement
39 exp -> ICONST
40 assignment_print_scan -> ID EQUAL exp
41 statement -> assignment_print_scan
42 statements -> statements statement
43 exp -> ICONST
44 exp -> ID
45 exp -> exp JOG exp
46 assignment_print_scan -> ID EQUAL exp
47 statement -> assignment_print_scan
48 statements -> statements statement
49 assignment_print_scan -> PRINT_KORI ID
50 statement -> assignment_print_scan
51 statements -> statements statement
52 func_def -> FUNC ID LPAREN RPAREN STARTT statements END
53 program -> func_def
54

```

Figure 2: Syntax Analysis Output

Intermediate Code

The generated intermediate code:

```

1 0: start -1
2 1: ld_int 12
3 2: store 0
4 3: ld_int 13
5 4: ld_var 0
6 5: add -1
7 6: store 1
8 7: print_int_value 1
9 8: halt -1

```

Listing 4: Intermediate Code

```

54
55 ===== INTERMEDIATE CODE=====
56 0: start -1
57 1: ld_int 12
58 2: store 0
59 3: ld_int 13
60 4: ld_var 0
61 5: add -1
62 6: store 1
63 7: print_int_value 1
64 8: halt -1
65

```

Figure 3: Intermediate Code

Assembly Code Output

The final x86 assembly code (excerpt):

```

1 .686
2 .model flat, c
3 .stack 100h
4 .data
5 output_integer_msg_format byte "%d", 0Ah, 0
6 .code
7 main proc
8     push ebp

```

```
9      mov ebp, esp
10     sub ebp, 100
11
12     ;ld_int 12
13     mov eax, 12
14     mov dword ptr [ebx], eax
15     add ebx, 4
16
17     ;store 0
18     mov dword ptr [ebp-0], eax
19
20     ;ld_int 13
21     mov eax, 13
22
23     ;ld_var 0
24     mov eax, [ebp-0]
25
26     ;add -1
27     add eax, edx
28
29     ;store 1
30     mov dword ptr [ebp-4], eax
31
32     ;print_int_value 1
33     INVOKE printf, ADDR output_integer_msg_format, eax
34
35     ret
36 main endp
37 end
```

Listing 5: Generated Assembly Code

5 Conclusion

This project successfully demonstrates the implementation of a complete compiler for a custom programming language. The compiler handles lexical analysis, syntax analysis, semantic checking, and code generation, producing functional x86 assembly code. The project provides valuable insights into compiler construction techniques and the complexities involved in language translation.

A Complete Source Code

A.1 Lexer.l - Lexical Analyzer

```

1 %option noyywrap
2 %{
3 #define INT_TYPE 1
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7 #include "parser.tab.h"
8 int lineno = 1;
9 void yyerror(char* s);
10 %}
11
12 alpha [a-zA-Z]
13 digit [0-9]
14 alnum {alpha}|{digit}
15 print [ -~]
16 ID {alpha}{alnum}*
17 ICONST [0-9]{digit}*
18
19 %%
20 "//" .* { }
21 "FUNC" { printf("Token: FUNC\n"); return FUNC; }
22 "start" { printf("Token: start\n"); return STARTT; }
23 "end" { printf("Token: end\n"); return END; }
24 "mone_kori" { printf("Token: mone_kori\n"); return MONE_KORI; }
25 "integer" { printf("Token: integer\n");
26             yylval.int_val=INT_TYPE; return INTEGER; }
27 "equal" { printf("Token: equal\n"); return EQUAL; }
28 "jog" { printf("Token: jog\n"); return JOG; }
29 "print_kori" { printf("Token: print_kori\n"); return PRINT_KORI; }
30 "<--" { printf("Token: <--\n"); return TYPE_ASSIGN; }
31 "(" { printf("Token: (\n"); return LPAREN; }
32 ")" { printf("Token: )\n"); return RPAREN; }
33 "{" { printf("Token: {\n"); return LBRACE; }
34 "}" { printf("Token: }\n"); return RBRACE; }
35 ";" { printf("Token: ;\n"); return SEMI; }
36 "=" { printf("Token: =\n"); return ASSIGN; }
37 {ID} { printf("Token: ID (%s)\n", yytext);
38         strcpy(yylval.str_val, yytext); return ID;}
39 {ICONST} { printf("Token: ICONST (%s)\n", yytext);
40             yylval.int_val=atoi(yytext); return ICONST;}
41 "\n" { lineno += 1; }
42 [ \t\r\f]+
43 . { yyerror("Unrecognized character"); }
44 %%

```

Listing 6: Complete Lexer Code

A.2 Parser.y - Syntax Analyzer

```

1 %{
2     #include <stdio.h>
3     #include <stdlib.h>
4     #include <string.h>
5     #include "syntab.c"
6     #include "codeGen.c"
7     void yyerror(char* s);
8     extern int lineno;
9     extern int yylex();
10    FILE *fp_syntax;
11 %}
12
13 %union
14 {
15     char str_val[100];
16     int int_val;
17 }
18
19 %token<int_val> FUNC STARTT END MONE_KORI INTEGER EQUAL JOG PRINT_KORI
20     TYPE_ASSIGN
21 %token ADDOP SUBOP MULOP DIVOP EQUOP LT GT
22 %token LPAREN RPAREN LBRACE RBRACE SEMI ASSIGN
23 %token<str_val> ID
24 %token<int_val> ICONST
25 %token<int_val> INT
26
27 %left LT GT /*LT GT has lowest precedence*/
28 %left JOG
29 %left MULOP /*MULOP has lowest precedence*/
30
31 %type<int_val> exp assignment_print_scan
32
33 %start program
34 %%
35 program: {gen_code(START, -1);} func_def {gen_code(HALT, -1); fprintf(
36     fp_syntax, "program -> func_def\n");}
37
38 func_def: FUNC ID LPAREN RPAREN STARTT statements END
39     {
40         fprintf(fp_syntax, "func_def -> FUNC ID LPAREN RPAREN
41             STARTT statements END\n");
42     }
43
44 statements: statements statement
45     {
46         fprintf(fp_syntax, "statements -> statements statement\n
47             ");

```

```

46     }
47     |
48     {
49         fprintf(fp_syntax, "statements -> empty\n");
50     };
51
52 statement: declaration
53     {
54         fprintf(fp_syntax, "statement -> declaration\n");
55     }
56     | assignment_print_scan
57     {
58         fprintf(fp_syntax, "statement -> assignment_print_scan\n");
59     }
60     ;
61
62 declaration: MONE_KORI ID TYPE_ASSIGN INTEGER
63     {
64         insert($2, $4);
65         fprintf(fp_syntax, "declaration -> MONE_KORI ID\n");
66     }
67     ;
68
69 assignment_print_scan: ID EQUAL exp
70     {
71         int address = idcheck($1);
72
73         if(address != -1)
74         {
75             gen_code(STORE, address);
76         }
77         else
78             yyerror("Undefined variable in assignment");
79
80         fprintf(fp_syntax, "assignment_print_scan -> ID\n");
81     }
82     | PRINT_KORI ID
83     {
84         int address = idcheck($2);
85
86         if(address != -1)
87         {
88             gen_code(PRINT_INT_VALUE, address);
89         }
90         else
91             yyerror("Undefined variable in print");
92
93         fprintf(fp_syntax, "assignment_print_scan ->

```

```

                                PRINT_KORI ID\n");
94         }
95         ;
96
97 exp: ICONST
98     {
99         gen_code(LD_INT, $1);
100        fprintf(fp_syntax, "exp -> ICONST\n");
101    }
102    | ID
103    {
104        int address = idcheck($1);
105
106        if(address != -1)
107        {
108            gen_code(LD_VAR, address);
109        }
110        else
111            yyerror("Undefined variable in expression");
112
113        fprintf(fp_syntax, "exp -> ID\n");
114    }
115    | exp JOG exp
116    {
117        gen_code(ADD, -1);
118        fprintf(fp_syntax, "exp -> exp JOG exp\n");
119    }
120    | exp MULOP exp
121    {
122        fprintf(fp_syntax, "exp -> exp MULOP exp\n");
123    }
124    | exp GT exp
125    {
126        fprintf(fp_syntax, "exp -> exp GT exp\n");
127    }
128    | exp LT exp
129    {
130        fprintf(fp_syntax, "exp -> exp LT exp\n");
131    }
132    ;
133
134 %%
135
136 void yyerror (char* s)
137 {
138     printf("Syntax error at line %d\n", lineno);
139     exit(1);
140 }
141
142 int main (int argc, char *argv[])
143 {

```

```
144     fp_syntax = fopen("syntax_log.txt", "w");
145     if (!fp_syntax) { printf("Error opening syntax log file\n");
146         return 1; }
147
148     printf("===== LEXICAL ANALYSIS =====\n");
149     yyparse();
150     fclose(fp_syntax);
151
152     printf("\nParsing finished!\n");
153
154     printf("\n===== SYNTAX ANALYSIS (Parse Tree)
155     =====\n");
156     fp_syntax = fopen("syntax_log.txt", "r");
157     if (fp_syntax) {
158         char ch;
159         while ((ch = fgetc(fp_syntax)) != EOF) putchar(ch);
160         fclose(fp_syntax);
161     }
162
163     printf("\n===== INTERMEDIATE CODE =====\n");
164     print_code();
165
166     printf("\n===== ASM CODE =====\n");
167     print_assembly();
168
169     return 0;
170 }
```

Listing 7: Complete Parser Code